



Topic:

Http Request and Response

Presented By:

AQSA BIBI & ALEESHA MAZHAR

Presented To:

MAAM AREEJ ALI

Table of Contents:

- Definition of Http
- ❖ **Http Request**
 - Http Methods
 - URL
 - Components of URL
 - Example Request the server
 - Http Request Header
 - Request Body
- ❖ **Http Response**
 - Elements
 - ✓ Status code
 - ✓ Response header
 - ✓ Response body

Http:

Http is an application layer protocol used for transmitting hypertext document, such as html over the internet. It is the foundation of data communication for the world wide web.



http://www

How does Http protocol work's ?

Hypertext Transfer Protocol (Http) is a method of encoding and transporting information between a client (such as a web browser) and a web server. Http is the primary protocol for transmission of information across the internet

Http follows a request-response paradigm in which clients makes a request and the server issues a response that include not only the requested contents, but also relevant status information about the request.

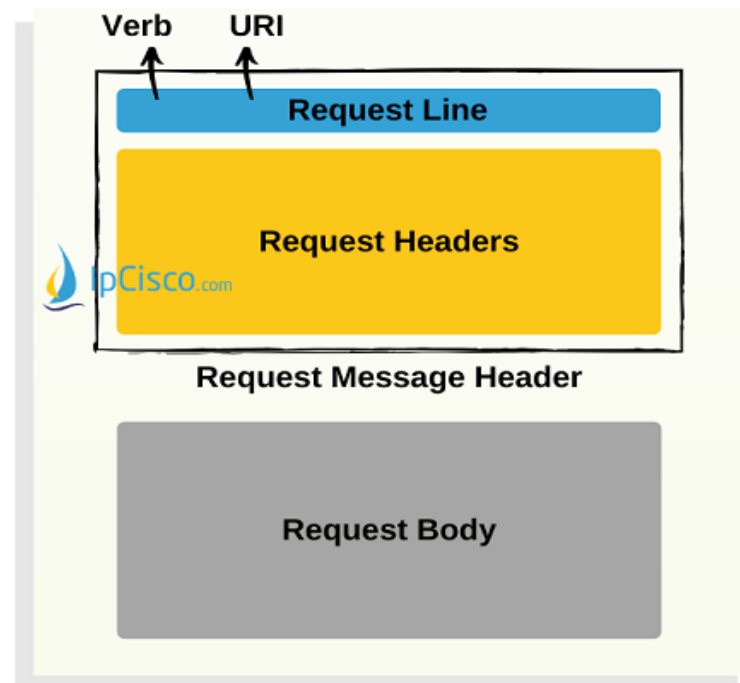
HTTP Request:

Http request is made by clients (typically a web browser) to a server, requesting a particular action or resource.

To make valid request, the client needs 4 basic components:

- URL (Uniform Resource Locator)
- Methods
- List of headers
- Body

HTTP Request Message



HTTP Methods:

The methods request tells the server what kind of action the client want the server to take.

Following are the http methods are:

GET:

GET request is used to read/retrieve data from a web server. GET returns an HTTP status code of 200 (OK) if the data is successful retrieved from the server.

POST:

POST request is used to send data (file, form data, etc) to the server. On the successful creation, it returns an HTTP status code of 201.

PUT:

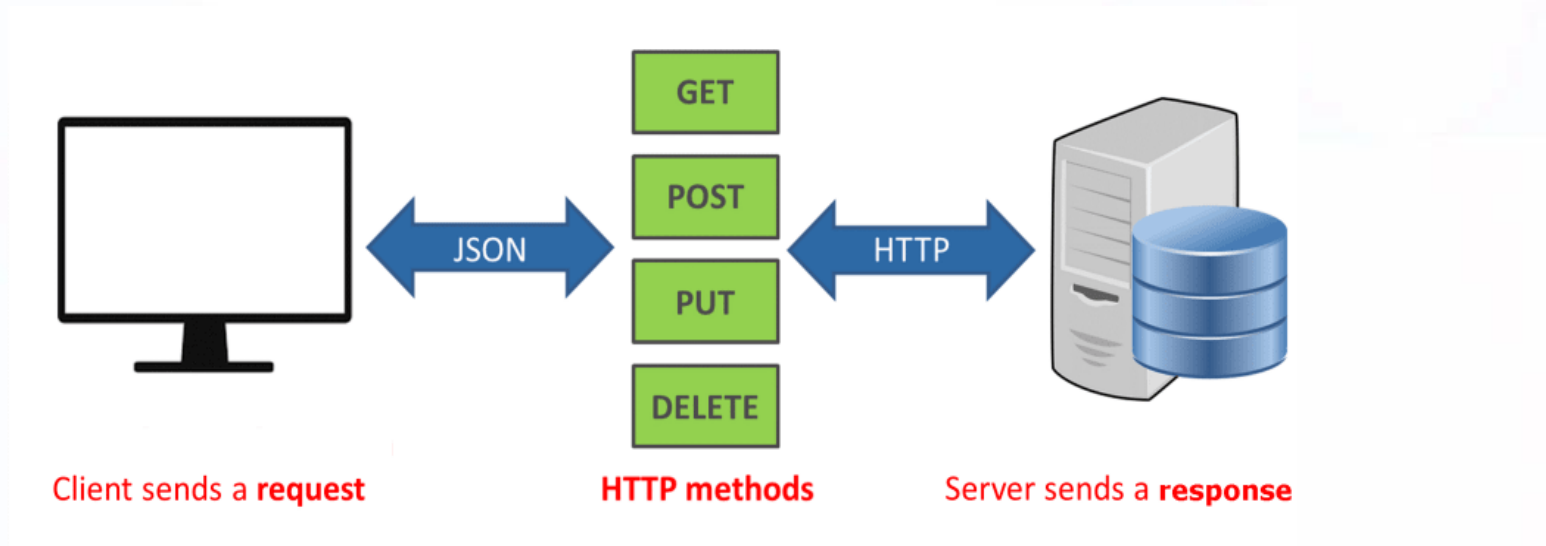
A PUT request is used to modify the data to the server. It replaces the entire content at a particular location with data that is passed in the body payload. If there are no resource that match the request, it will generate one.

PATCH:

PATCH is similar to PUT request, but the only difference is, it modifies a part of the data. It will only replace the content that you want to update.

DELETE:

A DELETE request is used to delete the data on the server at a specified location.



Example of HTTP request methods:

/books

GET	/books	Lists all the books in the database
DELETE	/books/{bookId}	Deletes a book based on their id
POST	/books	Creates a Book
PUT	/books/{bookId}	Method to update a book
GET	/books/{bookId}	Retrieves a book based on their id

URL:

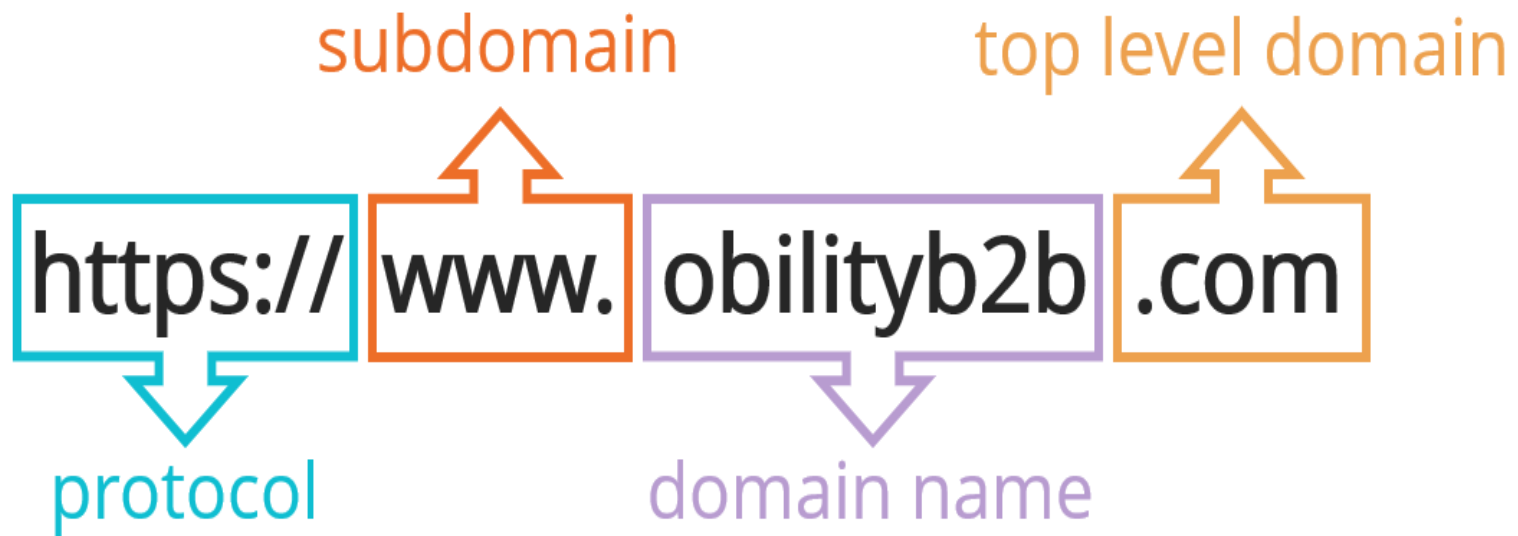
A URL (Uniform Resource Locator) is a unique identifier used to locate a resource on the internet. It is also referred to as a web address. URL consist of multiple parts- including a protocol and domain name- that tell a web browser how and where to retrieve a resource.

URL becomes an easy way for the client to tell the server which things it want to interact called resources.

For example:

A typical URL could have the form <http://www.example.com> which indicates a protocol (http) , a hostname(www.Example.com).

An example of URL:



Components of URL:

1. Protocols:

<http://www.example.com> it uses the https protocol HTTP (Hypertext Transfer Protocol) and HTTPS (Hypertext Transfer Protocol Secure) are the network communication protocols between web servers and web browser. They transmit messages, retrieve information through the domain name system (DNS) , and return it to the browser.

2. Subdomain:

It uses the www subdomain. A subdomain consists of any words or phrases that come before a URL's first dot. Referring to the world wide web, www is the most common type. It indicates that a website is accessible through the internet and uses HTTP to communicate.

3. Domain name:

It uses hostinger.com. The name of site you are visiting. Domain extension: It's the part following a domain name, for example, .com and .org.

4. Path:

Path specifies where you are on the website.

Example request the server:

The following example requests the server to save the given entity-body in hello.htm at the root of the server:

PUT /hello.htm HTTP/1.1

User-Agent: Mozilla/4.0 (compatible;
MSIE5.01;Window NT)

Host: www.example.com

Accept-Language: en-us

Connection: Keep-Alive

Content-type: text/html

Content-length: 182

HTTP Request Header:

HTTP Header play a crucial role in server and client behavior throughout the request and response cycle. Request header are sent by the client to the server and contain information and instructions related to the requested resource.

Some of the most commonly used request headers are:

Cookies:

Cookies contain the list of cookies on the client/browser, typically created by the server previously via a set-cookie response header.

User-Agent:

User-agent contains the information of the client/browser used by the user.

Accept-encoding:

Accept-encoding contains a list of the compression algorithms supported by the client, i.e. those that the server can use if it is configured to compress the response.

Accept-Language:

Accept-Language contains a list of languages supported by the server.

http://sterkinekor.com/

Request Header Name	Request Header Value
Host	sterkinekor.com
User-Agent	Specify any value here :)
Accept	text/html,application/xhtml+xml,application/
Accept-Language	en-us,en;q=0.5
Accept-Encoding	gzip, deflate
Connection	keep-alive
Cookie	__utma=125895759.1547992040.1345212987.13

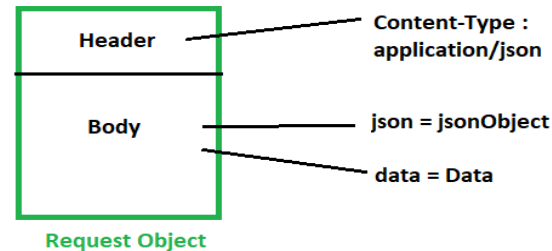
HTTP Request Body:

The request body contains the data the clients wants to send the server. PUT and POST are two methods that contain request body.

For example:

NAME= ALEENA&ORG= PERF

EASY&EXP=10YRS&SKILL=PERFORMANCE.



HTTP Request header and body example:

```
GET /doc/test.html HTTP/1.1
```

```
Host: www.test101.com
```

```
Accept: image/gif, image/jpeg, */*
```

```
Accept-Language: en-us
```

```
Accept-Encoding: gzip, deflate
```

```
User-Agent: Mozilla/4.0
```

```
Content-Length: 35
```

```
bookId=12345&author=Tan+Ah+Teck
```

Request Line

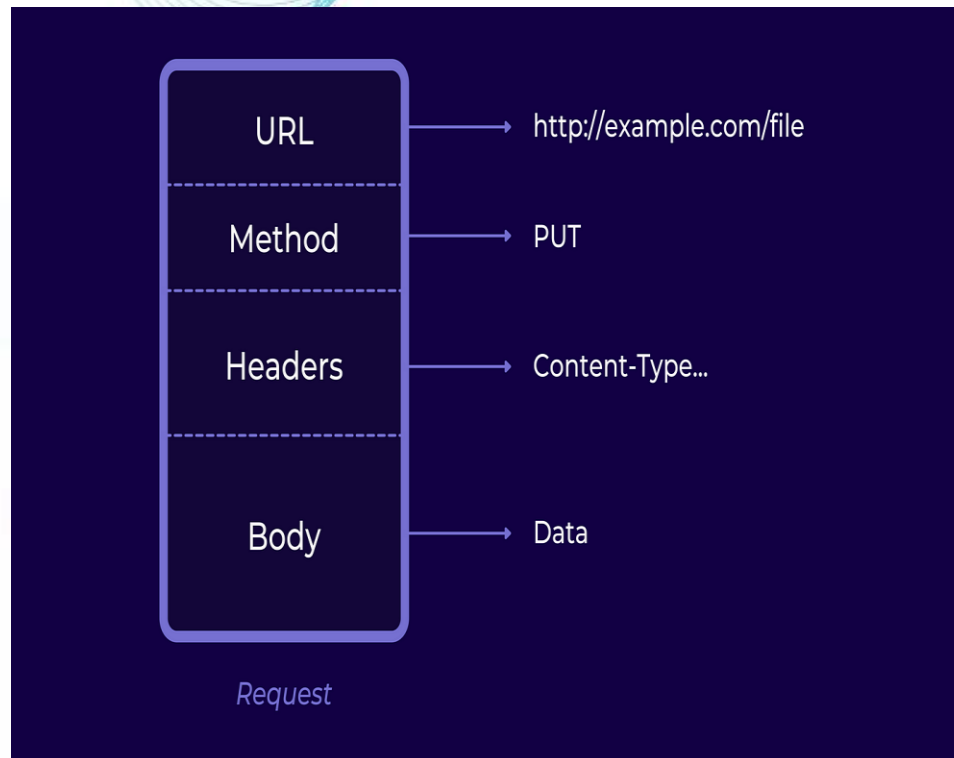
Request Headers

Request
Message
Header

A blank line separates header & body

Request Message Body

Http Request Example:



Example 2:

HTTP Method. { **POST** /apex/aws/rest/sdsrest/sample HTTP/1.1

Request Headers {
Host: www.example.com
Authorization: Basic
wY2nll1ruNNOExxOAJH7lg...:OJAdMS0r9OP_84yCn7ibQw..
Content-Type: application/json
Cache-Control: no-cache
XX-CLIENT-SYSTEM: Test System

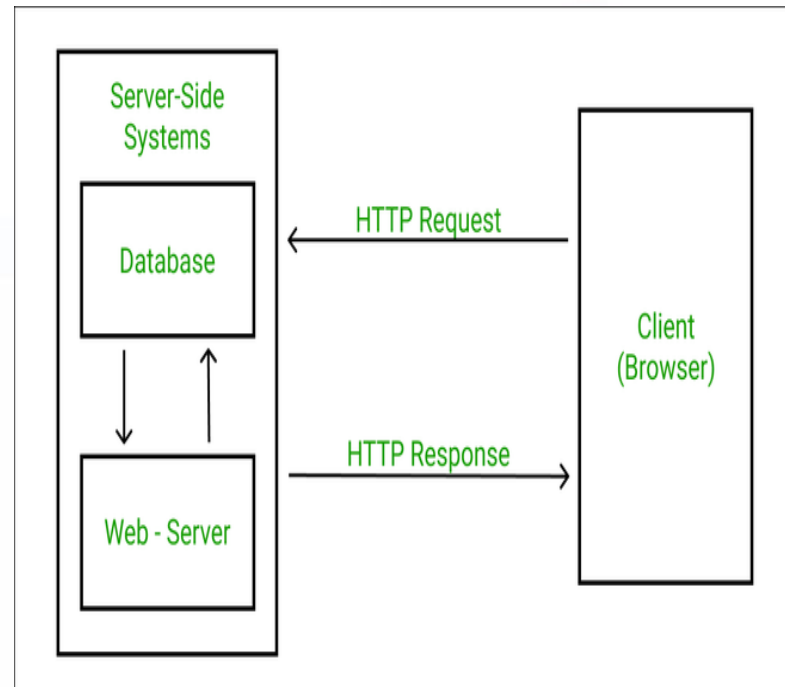
Request Body {
{
 color: "black",
 value: "#000"
}

Path to Resource

Protocol

HTTP Response

An HTTP response is made by a server to a client. The aim of the response is to provide the client with the resource it requested, or inform the client that the action it requested has been carried out; or else to inform the client that an error occurred in processing its request.



Elements of HTTP Response:

The response that the web server sends to the client consists of the following elements:

1. Status Code (e.g. 200 - OK, or 404 - Page Not Found, etc)
2. Response Headers
3. Response Body

HTTP/1.1 200 OK

Date: Sun, 08 Feb xxxx 01:11:12 GMT

Server: Apache/1.3.29 (Win32)

Last-Modified: Sat, 07 Feb xxxx

ETag: "0-23-4024c3a5"

Accept-Ranges: bytes

Content-Length: 35

Connection: close

Content-Type: text/html

<h1>My Home page</h1>

→ Status Line

} Response Headers

} Response Message Header

→ A blank line separates header & body

} Response Message Body

1. Status Code:

HTTP response status codes

HTTP response status codes indicate whether a specific HTTP request has been successfully completed.

Responses are grouped in five classes:

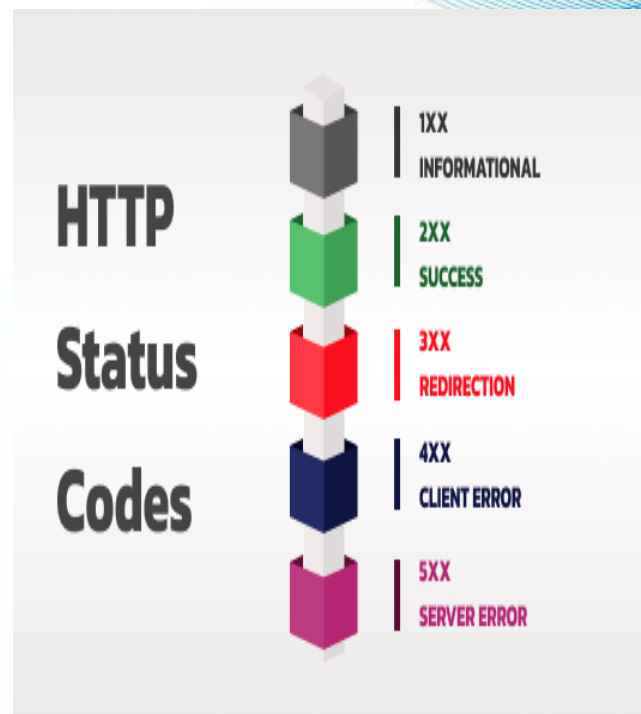
Informational responses (100 – 199)

Successful responses (200 – 299)

Redirection messages (300 – 399)

Client error responses (400 – 499)

Server error responses (500 – 599)



1xx Information responses:

An informational response indicates that the request was received and understood.

=====

1.100 Continue :

This interim response indicates that the client should continue the request or ignore the response if the request is already finished.

2.101 Switching Protocols:

This code is sent in response to an Upgrade request header from the client and indicates the protocol the server is switching to.

3.102 Processing (WebDAV):

This code indicates that the server has received and is processing the request, but no response is available yet.

for example:

HTTP/1.1 100 continue



2xx Successful responses:

This class of status codes indicates the action requested by the client was received, understood, and accepted.

=====

1. 200 OK:

The request succeeded. The result meaning of "success" depends on the HTTP method:

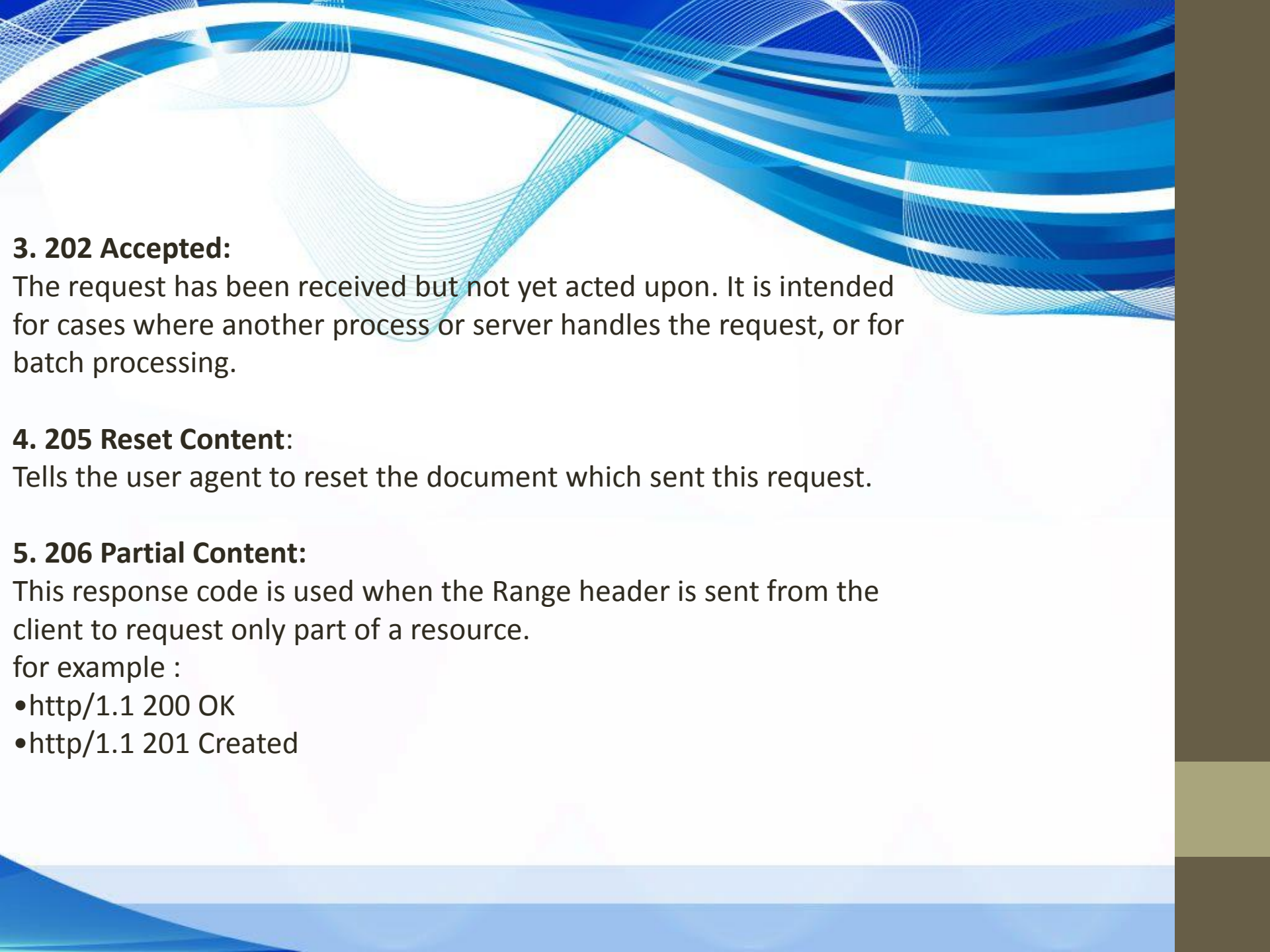
GET: The resource has been fetched and transmitted in the message body.

PUT or POST: The resource describing the result of the action is transmitted in the message body.

2. 201 Created :

The request succeeded, and a new resource was created as a result. This is typically the response sent after POST requests, or some PUT requests.





3. 202 Accepted:

The request has been received but not yet acted upon. It is intended for cases where another process or server handles the request, or for batch processing.

4. 205 Reset Content:

Tells the user agent to reset the document which sent this request.

5. 206 Partial Content:

This response code is used when the Range header is sent from the client to request only part of a resource.

for example :

- http/1.1 200 OK
- http/1.1 201 Created

3xx Redirection messages:

This class of status code indicates the client must take additional action to complete the request.

=====

1. 300 Multiple Choices:

The request has more than one possible response. The user agent or user should choose one of them.

2. 301 Moved Permanently:

The URL of the requested resource has been changed permanently. The new URL is given in the response.



3.302 Found:

This response code means that the URI of requested resource has been changed temporarily. Further changes in the URI might be made in the future. Therefore, this same URI should be used by the client in future requests.

4.304 Not Modified:

This is used for caching purposes. It tells the client that the response has not been modified, so the client can continue to use the same cached version of the response.



4xx Client error responses:

This class of status code is intended for situations in which the error seems to have been caused by the client.

=====

1. 400 Bad Request:

The server cannot or will not process the request due to something that is perceived to be a client error.

2. 401 Unauthorized:

This response means "unauthenticated". That is, the client must authenticate itself to get the requested response.

3. 402 Payment Required Experimental:

This response code is reserved for future use. The initial aim for creating this code was using it for digital payment systems, however this status code is used very rarely.



4. 403 Forbidden:

The client does not have access rights to the content; that is, it is unauthorized, so the server is refusing to give the requested resource.

5. 404 Not Found:

The server cannot find the requested resource. In the browser, this means the URL is not recognized. In an API, this can also mean that the endpoint is valid but the resource itself does not exist.



5xx Server error responses:

The server failed to fulfill a request.

Response status codes beginning with the digit "5" indicate cases in which the server is aware that it has encountered an error or is otherwise incapable of performing the request.

=====

1.500 Internal Server Error:

The server has encountered a situation it does not know how to handle.

2.501 Not Implemented:

The request method is not supported by the server and cannot be handled.

3. 502 Bad Gateway:

This error response means that the server, while working as a gateway to get a response needed to handle the request, got an invalid response.



4. 503 Service Unavailable:

The server is not ready to handle the request. Common causes are a server that is down for maintenance or that is overloaded.

5.504 Gateway Timeout:

This error response is given when the server is acting as a gateway and cannot get a response in time.

6.505 HTTP Version Not Supported:

The HTTP version used in the request is not supported by the server.



HTTP STATUS CODES

2xx Success

200 Success / OK

3xx Redirection

301 Permanent Redirect

302 Temporary Redirect

304 Not Modified

4xx Client Error

401 Unauthorized Error

403 Forbidden

404 Not Found

405 Method Not Allowed

5xx Server Error

501 Not Implemented

502 Bad Gateway

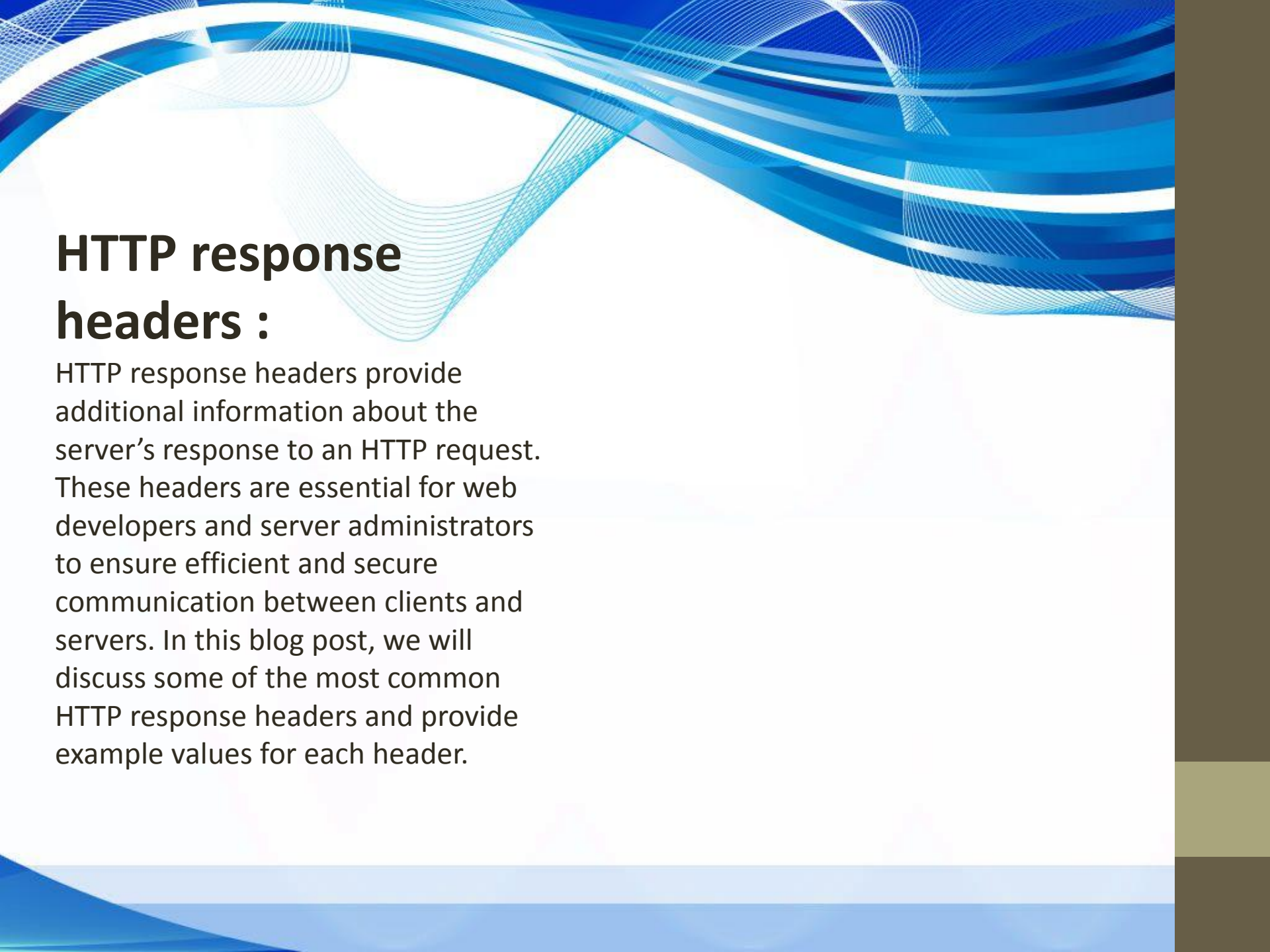
503 Service Unavailable

504 Gateway Timeout

Example of Status Code

Another Example of status code:

1XX	Informational codes	The server acknowledges and is processing the request.
2XX	Success codes	The server successfully received, understood, and processed the request.
3XX	Redirection codes	The server received the request, but there's a redirect to somewhere else (or, in rare cases, some additional action other than a redirect must be completed).
4XX	Client error codes	The server couldn't find (or reach) the page or website. This is an error on the site's side.
5XX	Server error codes	The client made a valid request, but the server failed to complete the request.

The background of the slide features a complex, abstract design of overlapping, wavy lines in various shades of blue and white. These lines create a sense of motion and depth, particularly concentrated in the upper half of the image. The lower half is mostly white, with a solid blue horizontal band at the very bottom. On the right side, there is a vertical strip of solid brown color.

HTTP response headers :

HTTP response headers provide additional information about the server's response to an HTTP request. These headers are essential for web developers and server administrators to ensure efficient and secure communication between clients and servers. In this blog post, we will discuss some of the most common HTTP response headers and provide example values for each header.

Response Headers:

Content-Type:

This header specifies the MIME type of the content being sent in the response. The MIME type tells the browser or other client how to interpret the content.

Example 1: Content-Type: text/html

Example 2: Content-Type: application/json

Example 3: Content-Type: image/png

Content-Length:

This header specifies the length of the content being sent in the response. This can be useful for the client to know how much data to expect and for the server to know how much data to send. When you are downloading a file from internet, the browser looks at this header to determine how big the file is.

Example 1: Content-Length: 1024

Example 2: Content-Length: 2048

Example 3: Content-Length: 4096

Set-Cookie:

This header is used to set a cookie in the client's browser. Cookies are small pieces of data that can be used to store information about the user or their session. The Set-Cookie header can include a name, value, expiration date, and other parameters.

Example

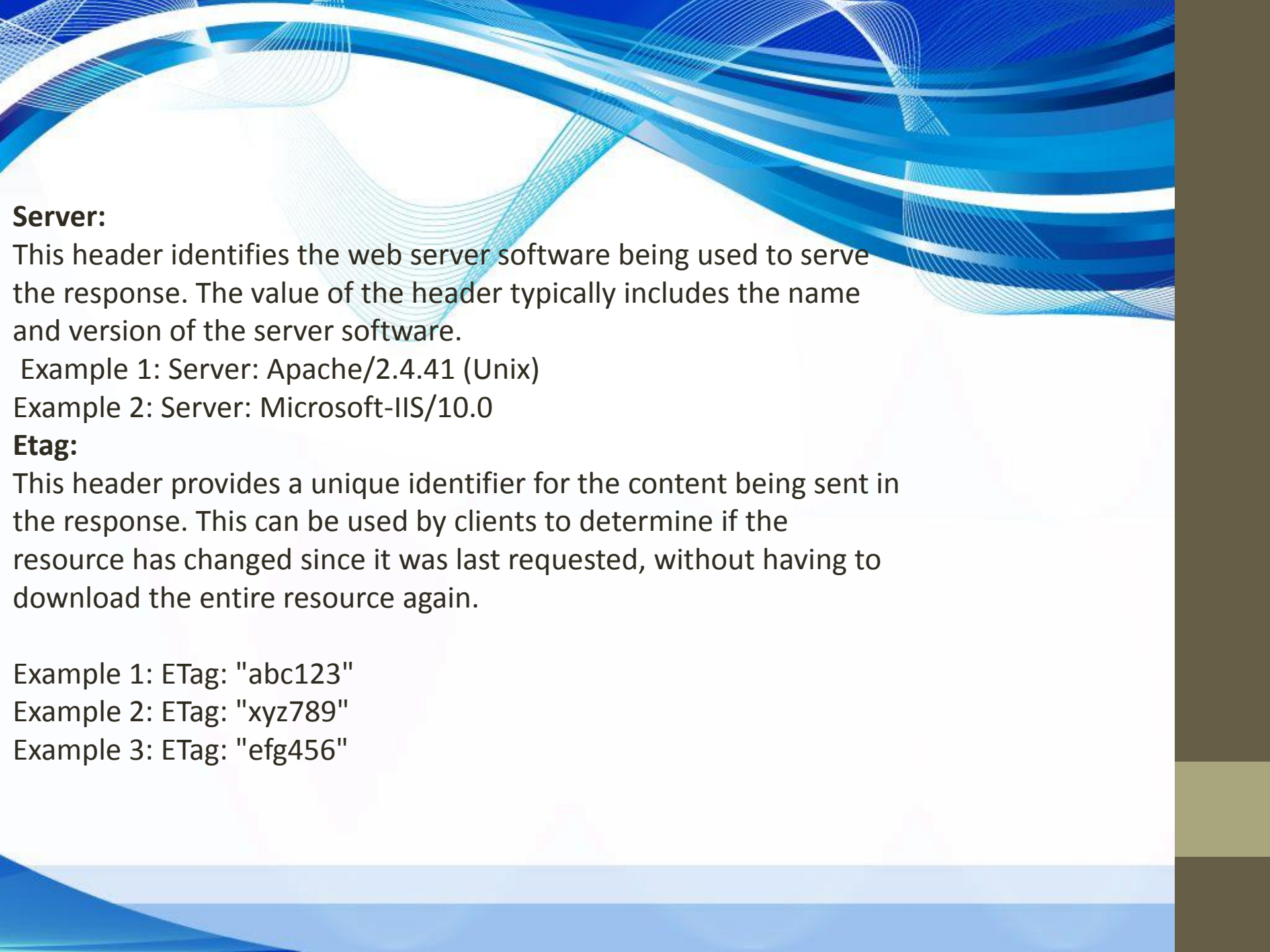
```
1: Set-Cookie: sessionId=12345; Expires=Thu, 01 Jan 2024  
00:00:00 GMT  
Domain=example.com
```

Access-Control-Allow-Origin:

This header is used to specify which domains are allowed to access the resource being requested, in the case of cross-origin requests. This can help prevent malicious scripts from accessing resources that they should not have access to.

Example 1: Access-Control-Allow-Origin: <https://www.example.com>

Example 2: Access-Control-Allow-Origin: *



Server:

This header identifies the web server software being used to serve the response. The value of the header typically includes the name and version of the server software.

Example 1: Server: Apache/2.4.41 (Unix)

Example 2: Server: Microsoft-IIS/10.0

Etag:

This header provides a unique identifier for the content being sent in the response. This can be used by clients to determine if the resource has changed since it was last requested, without having to download the entire resource again.

Example 1: ETag: "abc123"

Example 2: ETag: "xyz789"

Example 3: ETag: "efg456"



Last-Modified:

This header specifies the date and time when the content being sent in the response was last modified. This can be used by clients to determine if the resource has changed since it was last requested.

Example 1: Last-Modified: Tue, 04 May 2023 16:00:00 GMT

Example 2: Last-Modified: Wed, 03 May 2023 10:30:00 GMT

X-Frame-Options

This header is used to prevent clickjacking attacks by restricting which sites are allowed to embed the page in an iframe. The value of the header can include “deny” to prevent embedding on any site, “sameorigin” to allow embedding only on the same domain, or a specific domain to allow embedding on that domain only.

Example 1: X-Frame-Options: SAMEORIGIN

Example 2: X-Frame-Options: DENY

Example 3: X-Frame-Options: ALLOW-
FROM <https://www.example.com>





Location:

This header is used to redirect the client to a different URL. The client should automatically follow the URL in the Location header. This is commonly used for HTTP redirects, such as when a user submits a form and is redirected to a confirmation page.

Example 1: Location: <https://www.example.com/newpage>

Example 2: Location: /error.html

Example 3:

Location: <https://www.example.com/redirected>

Conclusion:

In conclusion, HTTP response headers provide valuable information about the data being sent in the response from the server to the client.



Http Response Header Example:

HTTP/1.1 200 OK

Access-Control-Allow-Origin: *

Connection: Keep-Alive

Content-Encoding: gzip

Content-Type: text/html; charset=utf-8

Date: Wed, 10 Aug 2016 13:17:18 GMT

Etag: "d9b3b803e9a0dc6f22e2f20a3e90f69c41f6b71b"

Keep-Alive: timeout=5, max=999

Last-Modified: Wed, 10 Aug 2016 05:38:31 GMT

Server: Apache

Set-Cookie: csrftoken=.....

Transfer-Encoding: chunked

Vary: Cookie, Accept-Encoding

X-Frame-Options: DENY

Response headers

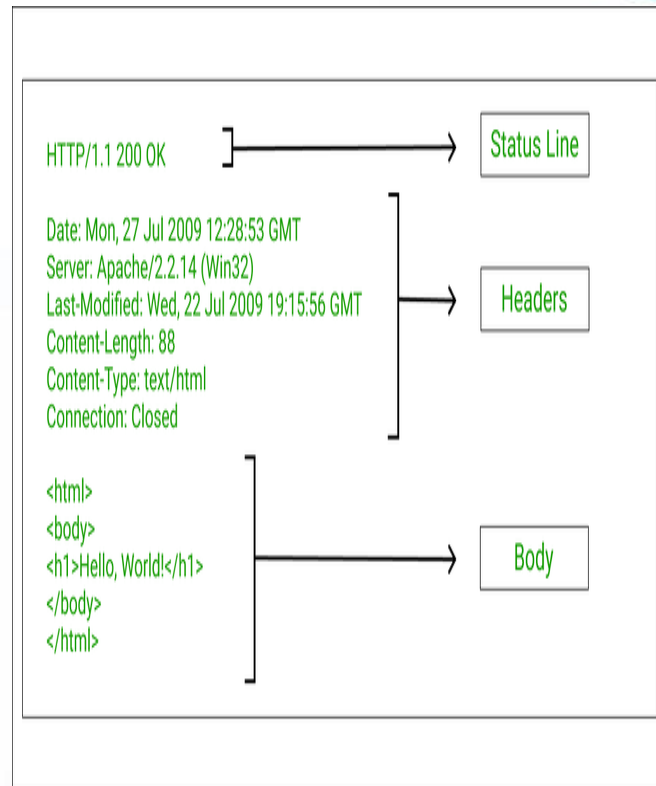
Entity headers

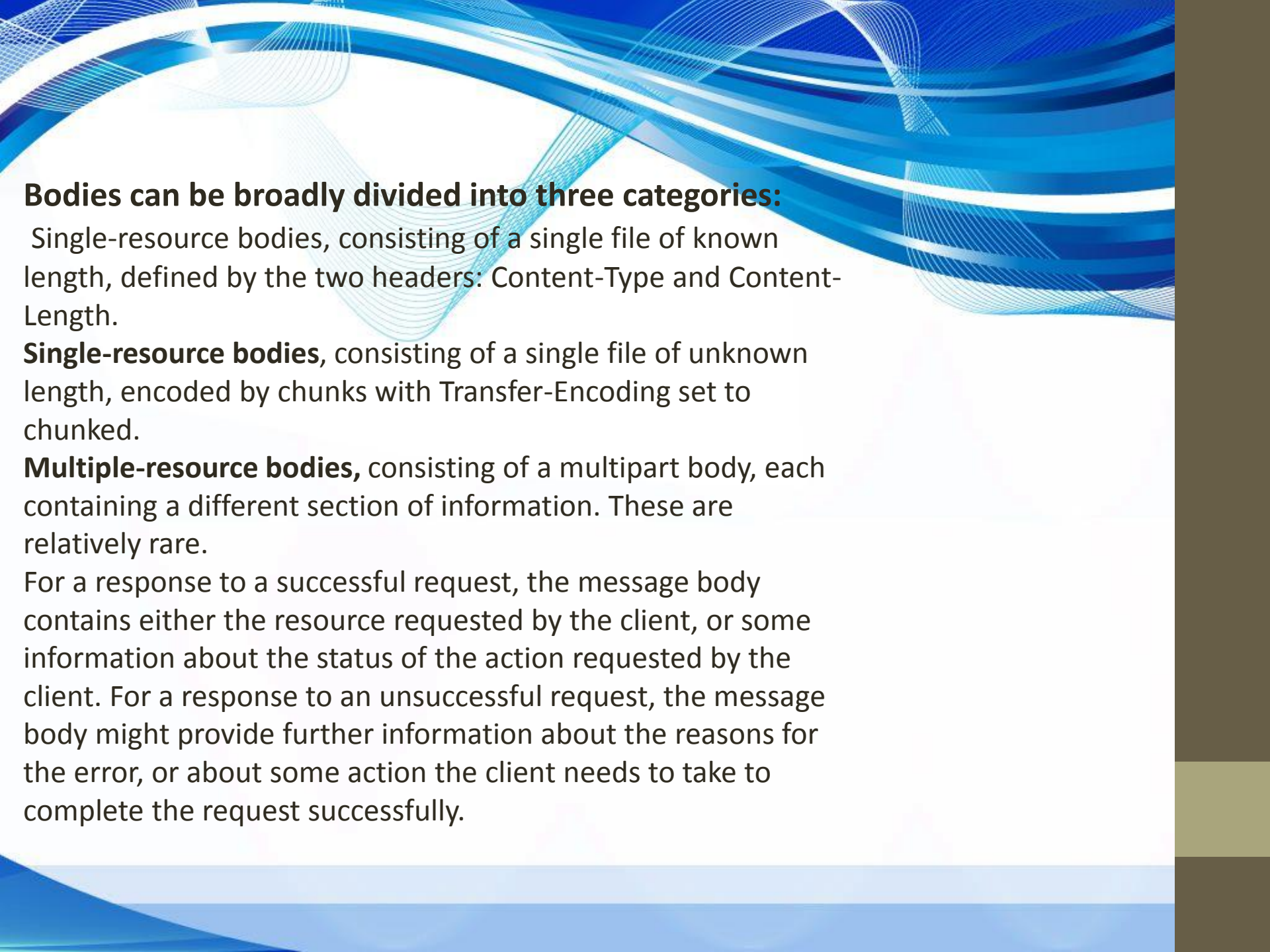
General headers

(body)

Response Body:

The response body is the content that is transmitted from the server to the client, which is what the client typically displays on the screen. It can be in various formats: HTML, text, GIF image, JPG (with EXIF data), WAV, MP4, etc., and is opened by the client/browser using the media type information contained in the Content-Type response header



The background of the slide features a complex, abstract design of overlapping, wavy lines in various shades of blue and white. These lines create a sense of motion and depth, particularly concentrated in the upper half of the image. The overall aesthetic is modern and technical.

Bodies can be broadly divided into three categories:

Single-resource bodies, consisting of a single file of known length, defined by the two headers: Content-Type and Content-Length.

Single-resource bodies, consisting of a single file of unknown length, encoded by chunks with Transfer-Encoding set to chunked.

Multiple-resource bodies, consisting of a multipart body, each containing a different section of information. These are relatively rare.

For a response to a successful request, the message body contains either the resource requested by the client, or some information about the status of the action requested by the client. For a response to an unsuccessful request, the message body might provide further information about the reasons for the error, or about some action the client needs to take to complete the request successfully.

HTTP/1.1 200 OK

Status Line

Date: Thu, 20 May 2004 21:12:58 GMT

Connection: close

General Headers

Server: Apache/1.3.27

Accept-Ranges: bytes

Response Headers

Content-Type: text/html

Content-Length: 170

Last-Modified: Tue, 18 May 2004 10:14:49 GMT

Entity Headers

<html>

<head>

<title>Welcome to the Amazing Site!</title>

</head>

<body>

<p>This site is under construction. Please come
back later. Sorry!</p>

</body>

</html>

Message Body

HTTP
Response

Example of HTTP
Response

ସମାବେଶ

